Projeto – ENE0211 - Teoria da Informação

Compactador de arquivos por codificação de Huffman

Prof. Daniel Guerreiro e Silva

Este projeto deve ser realizado **obrigatoriamente em duplas**, no entanto, **a nota é individual**. Além do programa em si, é objeto de avaliação a compreensão que cada integrante demonstra do que o programa faz e a qualidade da análise experimental sobre dados que são exclusivos da sua dupla. O uso de ferramentas de inteligência artificial é permitido, mas deve ser **obrigatoriamente documentado**, vide as regras na Parte IV.

Parte I – Implementação

1. Implementar, em linguagem C, C++ ou Python, um compactador/descompactador de arquivos, **de interface exclusiva por linha de comando**, que utiliza a codificação de Huffman. **Não é permitido usar bibliotecas que já implementem o algoritmo de Huffman**.

2. No modo "compactação" (opção `-c` via linha de comando), o programa recebe um caminho para um arquivo qualquer e gera um novo arquivo compactado.

   (a) Considere que o arquivo a ser compactado é uma fonte binária sem memória com dígitos agrupados em blocos de tamanho 8; portanto, leia o arquivo byte a byte, contando as ocorrências de cada byte a fim de estimar sua respectiva probabilidade (frequência relativa).

   (b) Construa a árvore do código de Huffman e codifique o arquivo. Como as palavras-código têm tamanho variável (menor ou maior que 8 bits), será necessário fazer escrita bit-a-bit e *zero-padding* ao fim do arquivo, para gravar uma quantidade múltipla de um byte.

   (c) Ao final do processo, mostre na saída padrão a (*i*) entropia estimada da fonte ($\hat{H}$, em bits/símbolo, considerada na forma estendida, i.e. em blocos de 8 bits), (*ii*) o comprimento médio do código (L, em bits/símbolo) e (*iii*) a taxa de compressão (tam. arq. original / tam. arq. compactado).

3. No modo "descompactação" (opção `-d` via linha de comando), o programa lê o um caminho para um arquivo comprimido e gera um arquivo decodificado, equivalente ao formato original. O arquivo compactado deverá conter, além dos dados comprimidos, uma área de cabeçalho com informações que permitam reconstruir a árvore do código.

4. **Casos de borda (obrigatórios).** O programa deve tratar corretamente, sem travar e sem corromper os dados, as quatro situações abaixo. Cada uma delas será testada na correção:

   (a) arquivo de entrada **vazio** (0 bytes);

   (b) arquivo com **um único byte**;

   (c) arquivo contendo **um único símbolo distinto** repetido 10^6 vezes (atenção: a árvore de Huffman degenera neste caso — a dupla deve decidir e justificar a solução adotada);

   (d) arquivo em que todos os 256 valores de byte ocorrem com frequências muito próximas.

5. **Verificação de integridade.** Para todo arquivo testado, a dupla deve verificar que a descompactação reproduz o original *byte a byte* (por exemplo, com `diff` ou comparação de *hash* SHA-256) e registrar essa verificação no relatório.

Exemplo de uso no prompt do comando / terminal:

```
$$ meucompactador -c livro.txt
$$ meucompactador -d livro.txt.huff
```

Parte II – Corpus experimental (exclusivo da sua dupla)

Os ensaios devem ser feitos sobre um corpus próprio, montado por cada dupla e descrito a seguir. Os resultados numéricos serão, portanto, diferentes para cada dupla, e o relatório deve analisar os seus números.

II.1 – Fonte sintética com entropia conhecida

Escreva um pequeno programa gerador que produza um arquivo de 5×10^6 bytes, emitidos de forma independente por uma fonte sem memória com alfabeto $\mathcal{A} = \{0, 1, \ldots, k-1\}$ (valores possíveis de byte) e distribuição geométrica truncada

$$p_i \;=\; \frac{r^{\,i}\,(1-r)}{1-r^{\,k}}, \qquad i = 0, 1, \ldots, k-1.$$

Os parâmetros são determinados pelas matrículas da dupla. Seja

$$m \;=\; \big(\text{soma das matrículas dos dois integrantes}\big) \bmod 100,$$

e então

$$k \;=\; 4 + \big(m \bmod 5\big), \qquad r \;=\; 0{,}55 + 0{,}02 \cdot \Big(\big\lfloor m/5 \big\rfloor \bmod 10\Big).$$

Esta é a única fonte do corpus cuja entropia *teórica* é conhecida em forma fechada. Ela existe para que a dupla possa validar a própria implementação: se L não estiver no intervalo $[H, H+1)$, há um erro no programa. Informe no relatório os valores de m, k, r, a entropia teórica $H(X)$ calculada a partir da fórmula acima, e a entropia empírica $\hat{H}$ estimada pelas frequências observadas no arquivo gerado.

II.2 – Arquivos reais

1. um arquivo de **texto** em português, com pelo menos 1 MB, obtido de um acervo público (por exemplo, Domínio Público ou Project Gutenberg); indique a obra e a procedência;
2. uma **imagem** em formato BMP de 24 bits, sem compressão, **fotografada pela própria dupla**;
3. um **áudio** em formato WAV PCM, sem compressão, **gravado pela própria dupla**;
4. um arquivo **já comprimido** (JPEG, MP3, ou ZIP), à escolha da dupla;
5. os quatro casos de borda da Parte I, item 4.

Parte III – Relatório analítico

Elabore um relatório analítico de no máximo **8 páginas**, que deve conter:

III.1 – Tabela de resultados

Uma tabela com uma linha por arquivo do corpus e as colunas: tamanho original (bytes); $\hat{H}$ (bits/símbolo); L (bits/símbolo); tamanho do cabeçalho (bytes); tamanho total compactado (bytes); taxa de compressão; e a razão $L/\hat{H}$.

III.2 – Análise

Responda, com base nos *seus* números:

1. Verifique se $\hat{H} \leq L < \hat{H} + 1$ vale em todos os arquivos do corpus. Este resultado é uma consequência de qual teorema visto em aula? O que significaria, em termos de implementação, encontrar um caso em que a desigualdade falha?

2. Para a fonte sintética, compare a entropia teórica $H(X)$ com a entropia empírica $\hat{H}$. Elas coincidem? Explique a origem da diferença e estime como ela se comportaria se o arquivo tivesse 10^3 bytes em vez de 5×10^6.

3. A taxa de compressão observada é sistematicamente *menor* que a razão $8/L$, que é o limite superior da taxa de compressão teórica. Identifique e **quantifique separadamente**, em bytes, as três causas dessa perda em pelo menos dois arquivos do corpus: (*i*) o cabeçalho, (*ii*) o *zero-padding* final e (*iii*) o fato de os *comprimentos das palavras-código* serem inteiros, isto é, a redundância $L - \hat{H} > 0$.

4. Para o arquivo já comprimido (JPEG, MP3 ou ZIP), o arquivo de saída provavelmente ficou *maior* que o de entrada. Tente explicar o resultado em termos da entropia medida neste arquivo.

5. Para o arquivo de símbolo único repetido, descreva o que o algoritmo de Huffman produz literalmente, por que isso é um caso degenerado, qual solução a dupla adotou e qual é a taxa de compressão resultante. Compare essa taxa com a que seria teoricamente possível para essa fonte.

6. Para a imagem BMP, compare a taxa obtida pelo seu compactador com a que o formato PNG obtém sobre a mesma imagem. Explique a diferença em termos do modelo de fonte que cada um assume.

7. Ordene os arquivos do corpus por taxa de compressão obtida e explique a ordem observada em termos de **estrutura estatística** da fonte, e não apenas em termos de "tipo de arquivo".

Parte IV – Documentação do uso de ferramentas de IA

O uso de assistentes de IA (ChatGPT, Claude, Copilot, Gemini ou similares) é **permitido**. Em contrapartida, ele é de **declaração obrigatória**, em um anexo separado (arquivo à parte ou apêndice do relatório) contendo:

1. quais ferramentas foram utilizadas e em que partes do projeto;

2. os *prompts* efetivamente usados nas interações relevantes (pode ser um apanhado representativo, não é necessário o histórico completo);

3. **pelo menos três erros, imprecisões ou limitações técnicas** que vocês identificaram na saída da ferramenta, com a explicação de por que estavam errados e como foram corrigidos, este é o item de maior peso do anexo;

4. quais trechos do código e do relatório foram escritos integralmente pela dupla.

Observação: não declarar o uso de IA, quando ele ocorreu, é falta de integridade acadêmica e levará à nota zero no projeto. Se a dupla afirmar não ter identificado nenhum erro nas saídas da ferramenta, deverá descrever em detalhe quais verificações independentes realizou para chegar a essa conclusão.

Arguição oral

Cada dupla fará uma **arguição oral de aproximadamente 12 minutos**, em data a combinar após a entrega. As perguntas versarão **exclusivamente sobre o código e o relatório que a própria dupla entregou**: decisões de implementação, estrutura do cabeçalho, tratamento dos casos de borda, e os números que constam da sua tabela de resultados.

Pontos a observar:

- **a nota da arguição é individual**; os dois integrantes serão questionados separadamente sobre partes distintas do trabalho, e podem receber notas diferentes;
- cada integrante deve ser capaz de explicar **qualquer** parte do que foi entregue, e não apenas a parte que escreveu;
- não ser capaz de explicar o funcionamento do código entregue limita a nota final do projeto, independentemente de o programa funcionar corretamente;
- é permitido consultar o próprio código durante a arguição.
- **a ausência de um integrante na arguição implica nota zero do projeto para ele, mesmo que o outro integrante compareça.**

Entrega e avaliação

Entregáveis

Entregar via ambiente Aprender, em um único arquivo compactado:

1. o código-fonte do compactador e do gerador da fonte sintética;
2. `README` explicando configuração (bibliotecas), compilação e uso, contendo nome e matrícula dos dois integrantes;
3. o relatório analítico em PDF;
4. o anexo de documentação do uso de IA;
5. o **link de um repositório** (GitHub, GitLab ou similar) com o histórico de *commits* do desenvolvimento. O histórico não é pontuado, mas será consultado durante a arguição.

Os arquivos do corpus **não** devem ser enviados (exceto os casos de borda, se pequenos): indique no relatório como obtê-los ou reproduzi-los.

Composição da nota

Componente	**Peso**
Arguição oral (individual)	40 %
Relatório analítico	30 %
Corretude do programa (integridade da descompactação e casos de borda)	25 %
Documentação do uso de IA	5 %
Total	**100 %**

Data de entrega: 29/11/2026 até 23h59min.

Arguições: Aulas dos dias 1/12/2026 e 3/12/2026, em horário a combinar.